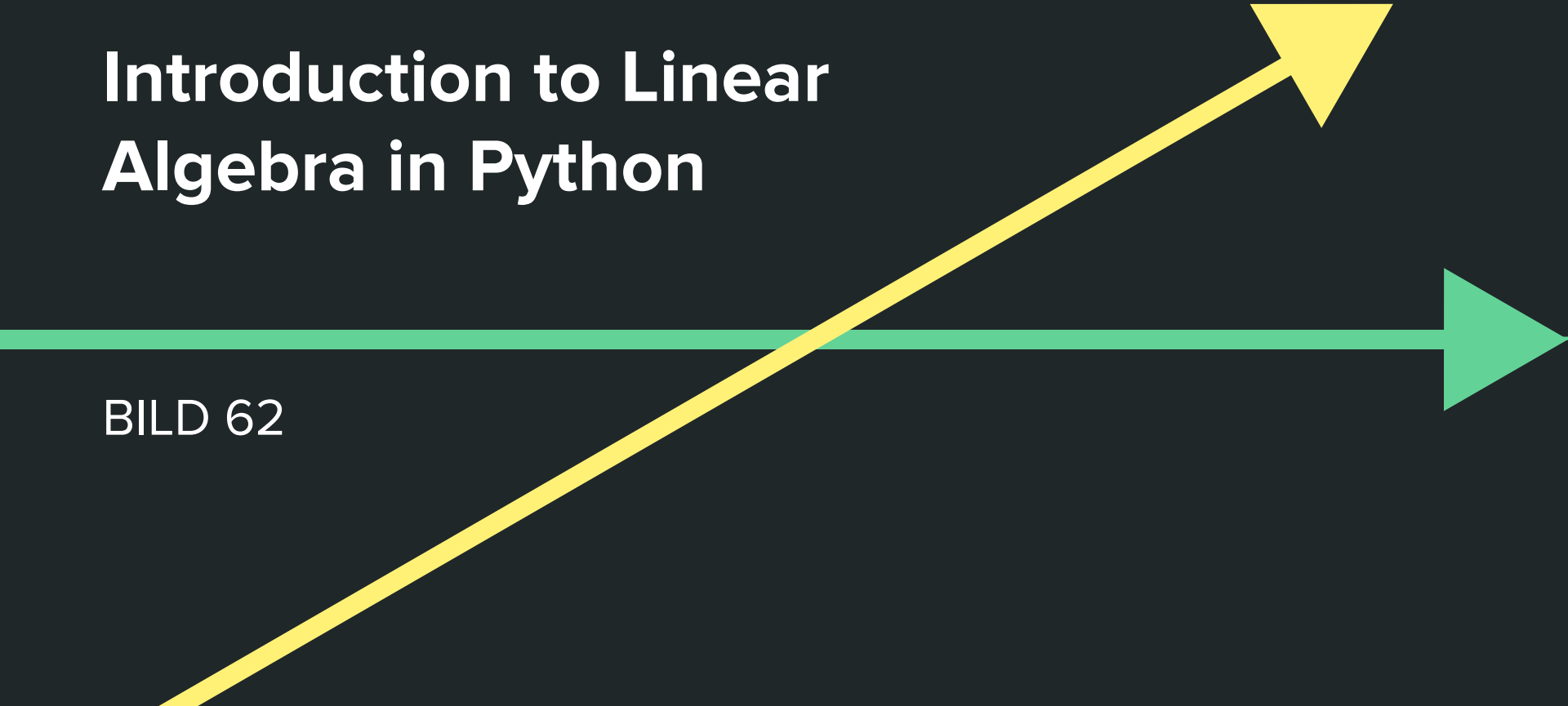


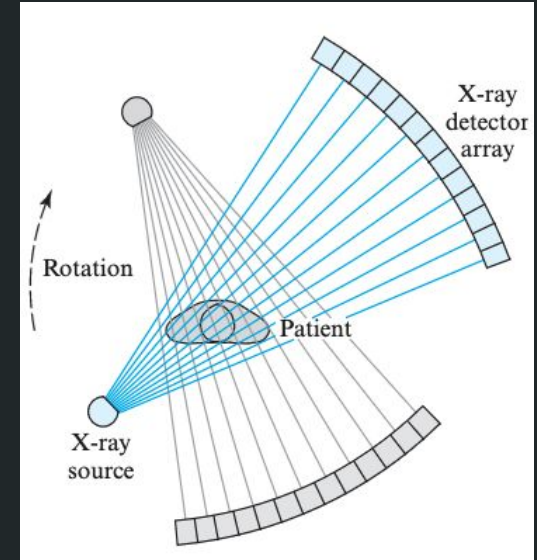
Introduction to Linear Algebra in Python

BILD 62



Why linear algebra?

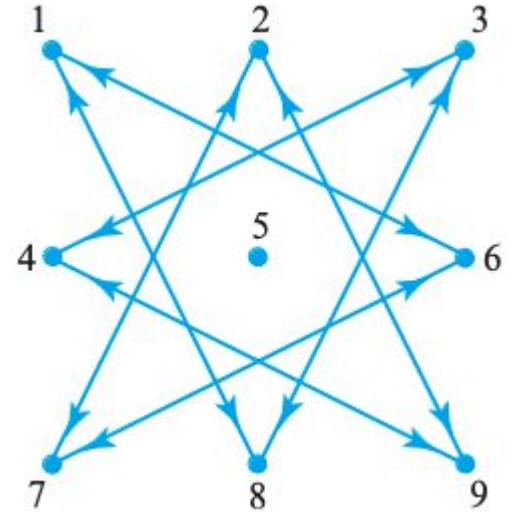
- The “language of data”!
- Useful in a variety of contexts, from simplifying large datasets to modeling populations of animals
- It's how we can find solutions to multiple **linear** equations



Linear algebra can be used to solve for angles in a CAT scan

Additional use cases of linear algebra in biology

- Modeling of population dynamics
- Analysis of food web dynamics & ecology
- Genetics & DNA sequencing (e.g., sequence alignment)
- Metabolic pathway analysis
- Dimensionality reduction!!!!
 - Protein structure & folding
 - Neurophysiology data
- Image analysis (convolution, matrix transformations)

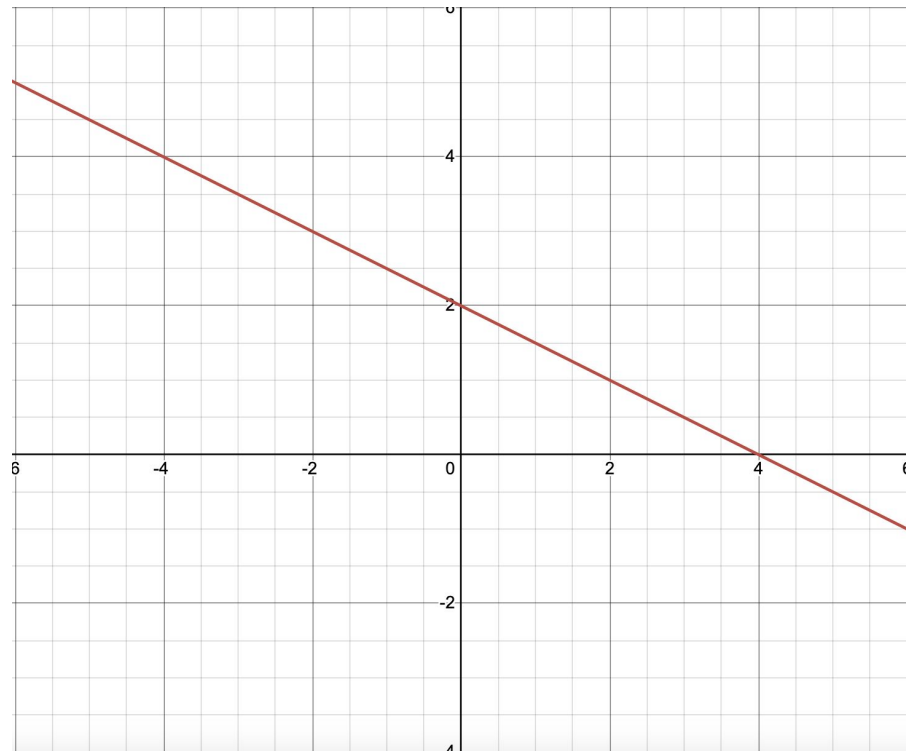


Linear Algebra is the mathematics of linear equations

- Slope-Intercept Form:

$$y = mx + b$$

- Describes the relationship between a dependent variable y and the independent variable x
- The equation has two parameters: the slope m and the intercept b
- All variables are raised to the first power



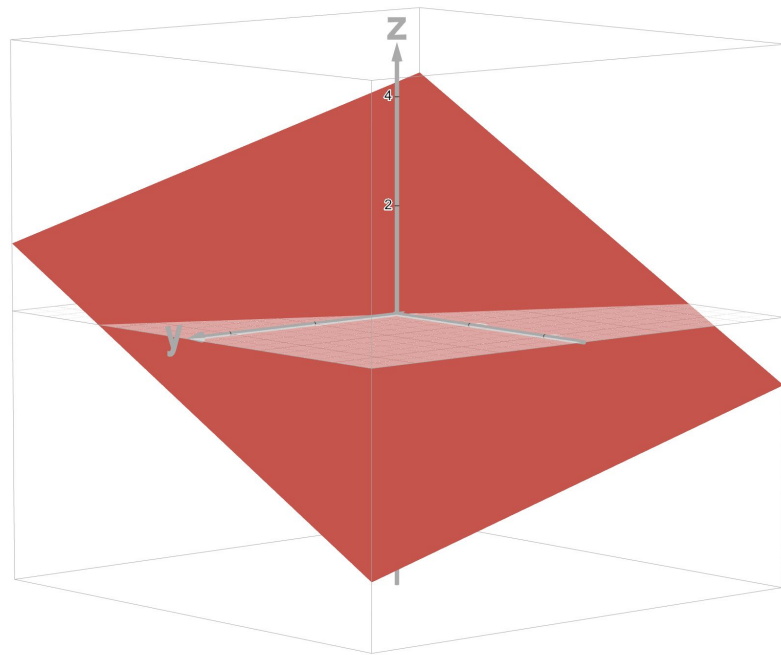
Linear Equations exist in higher dimensions too

- Consider the equation:

$$z = 0.5x - 0.25y + 0$$

- One dependent variable z and two independent variables x and y
- 2D line is a plane, 3D and higher are called hyperplanes
- An n -dimensional linear equation is expressed as:

$$a_1x_1 + a_2x_2 + a_3x_3 + \dots + a_{n-1}x_{n-1} + a_nx_n = b$$



Linear Algebra is useful for solving systems of linear equations

- Sometimes variables in your dataset are independent and are described best by a **system of equations**

- Dynamics of structured populations (e.g. children, adults)
- Neuron activity
- Biochemical Reactions

- If adults produce 2 children each year and:
 - 50% of children survive to adult hood
 - 80% of adults survive

$$C_{t+1} = 2A_t$$

$$A_{t+1} = 0.5C_t + 0.8A_t$$

- As you start dealing with more equations, this notation becomes unruly
- Before we perform any linear algebra, we have to put our equations in **matrix-vector** form

Suppose that a farmer has a large population of plants consisting of some distribution of all three possible genotypes AA , Aa , and aa . The farmer desires to undertake a breeding program in which **each plant in the population is always fertilized with a plant of genotype AA** and is then replaced by one of its offspring.

We want to derive an expression for the distribution of the three possible genotypes in the population after any number of generations.



Solving punnett squares over generations with linear algebra

Genotype of Offspring	Genotypes of Parents					
	<i>AA-AA</i>	<i>AA-Aa</i>	<i>AA-aa</i>	<i>Aa-Aa</i>	<i>Aa-aa</i>	<i>aa-aa</i>
<i>AA</i>	1	$\frac{1}{2}$	0	$\frac{1}{4}$	0	0
<i>Aa</i>	0	$\frac{1}{2}$	1	$\frac{1}{2}$	$\frac{1}{2}$	0
<i>aa</i>	0	0	0	$\frac{1}{4}$	$\frac{1}{2}$	1

probability of observing genotype

Formalizing our problem...

a_n = fraction of plants of genotype AA in n th generation

b_n = fraction of plants of genotype Aa in n th generation

c_n = fraction of plants of genotype aa in n th generation

One additional rule...

$$a_n + b_n + c_n = 1 \quad \text{for } n = 0, 1, 2, \dots$$

We can determine the genotype distribution of each generation from the genotype distribution of the *preceding* generation by the following system of linear equations:

$$a_n = a_{n-1} + \frac{1}{2}b_{n-1}$$

$$b_n = c_{n-1} + \frac{1}{2}b_{n-1}$$

$$c_n = 0$$

- 
1. All the offspring of a plant of genotype AA will be AA
 2. Half of the offspring of a plant of Aa will be AA

Putting this in the language of linear algebra

$$\mathbf{x}^{(n)} = M\mathbf{x}^{(n-1)}, \quad n = 1, 2, \dots$$

where

$$\mathbf{x}^{(n)} = \begin{bmatrix} a_n \\ b_n \\ c_n \end{bmatrix}, \quad \mathbf{x}^{(n-1)} = \begin{bmatrix} a_{n-1} \\ b_{n-1} \\ c_{n-1} \end{bmatrix}, \quad \text{and} \quad M = \begin{bmatrix} 1 & \frac{1}{2} & 0 \\ 0 & \frac{1}{2} & 1 \\ 0 & 0 & 0 \end{bmatrix}$$

coefficients matrix

First three columns of
our table, AND the
coefficients in our
three equations!

Example from [Elementary Linear Algebra](#)

Putting this in the language of linear algebra

$$\mathbf{x}^{(n)} = M\mathbf{x}^{(n-1)}, \quad n = 1, 2, \dots$$

where

$$\mathbf{x}^{(n)} = \begin{bmatrix} a_n \\ b_n \\ c_n \end{bmatrix}, \quad \mathbf{x}^{(n-1)} = \begin{bmatrix} a_{n-1} \\ b_{n-1} \\ c_{n-1} \end{bmatrix}, \quad \text{and} \quad M = \begin{bmatrix} 1 & \frac{1}{2} & 0 \\ 0 & \frac{1}{2} & 1 \\ 0 & 0 & 0 \end{bmatrix}$$

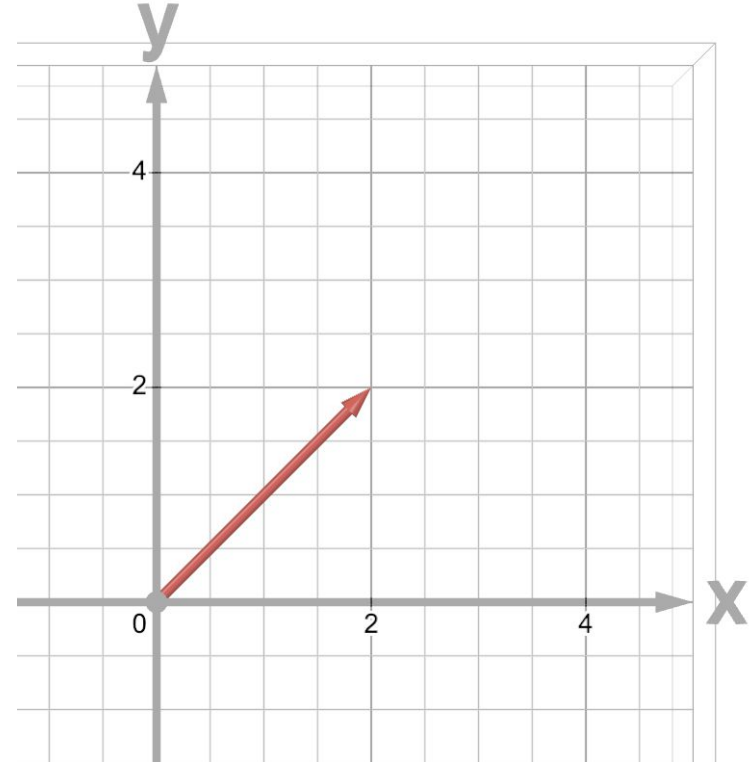
 

Example from [Elementary Linear Algebra](#)

Part I: Vectors and Vector Operations

Vectors contain multitudes

- Describe quantities that can't be described with a single number (scalar)
- Can be represented graphically as an arrow
 - The tip of the arrow is given by a list of numbers (coordinates)



Vectors contain multitudes

- Describe quantities that can't be described with a single number (scalar)
- Can be represented graphically as an arrow
 - The tip of the arrow is given by a list of numbers (coordinates)
- Vectors come in two main types/orientations:
 - Row vectors (1 row, many columns)

$$\mathbf{a} = [a_1 \quad a_2 \quad \dots \quad a_n]$$

Row Vector

Vectors contain multitudes

- Describe quantities that can't be described with a single number (scalar)
- Can be represented graphically as an arrow
 - The tip of the arrow is given by a list of numbers (coordinates)
- Vectors come in two main types/orientations:
 - Row vectors (1 row, many columns)
 - Column vectors (1 column, many rows)

$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_m \end{bmatrix}$$

Column Vector

Vector property: **Orientation**

whether the vector is in **column orientation (standing up tall)** or in **row orientation (flat and wide)**

Are these the same?
Does it matter?
Sometimes yes,
sometimes no.

column vector

$$\mathbf{x} = \begin{pmatrix} 4 \\ 1 \\ 2 \end{pmatrix}$$

We can move
between these with
transposition

row vector

$$\mathbf{x}^T = \begin{pmatrix} 4 & 1 & 2 \end{pmatrix}$$

Note: The convention in linear algebra is to *assume* vectors are in column orientation, unless otherwise specified.

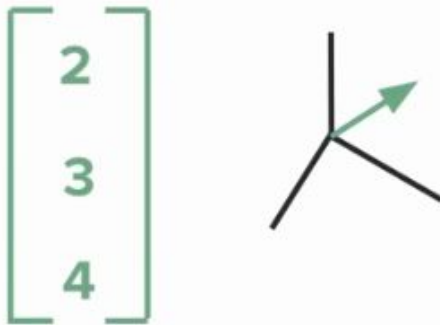
Vector property: **Dimensionality**

(the number of numbers in the vector)

Two dimensional



Three dimensional



N dimensional



Note: this is different than dimensionality in code!
Mathematical dimensionality is **length** or **shape** in Python.

We can create vectors in python with NumPy

- `np.array()` allows us to create vectors by using nested lists
- To create a column vector: `np.array([ [x1], [x2], ..., [xm] ])`
- To create a row vector: `np.array([ [a1, a2, ..., an] ])`

$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_m \end{bmatrix}, \quad \mathbf{a} = [a_1 \quad a_2 \quad \dots \quad a_n]$$

Let's open datahub!

Vector addition operates element-by-element

- You can add vectors with the same orientation and dimensionality
- When 2 d-dimensional vectors a and b are added together to produce a new vector c, each element in c is computed as:

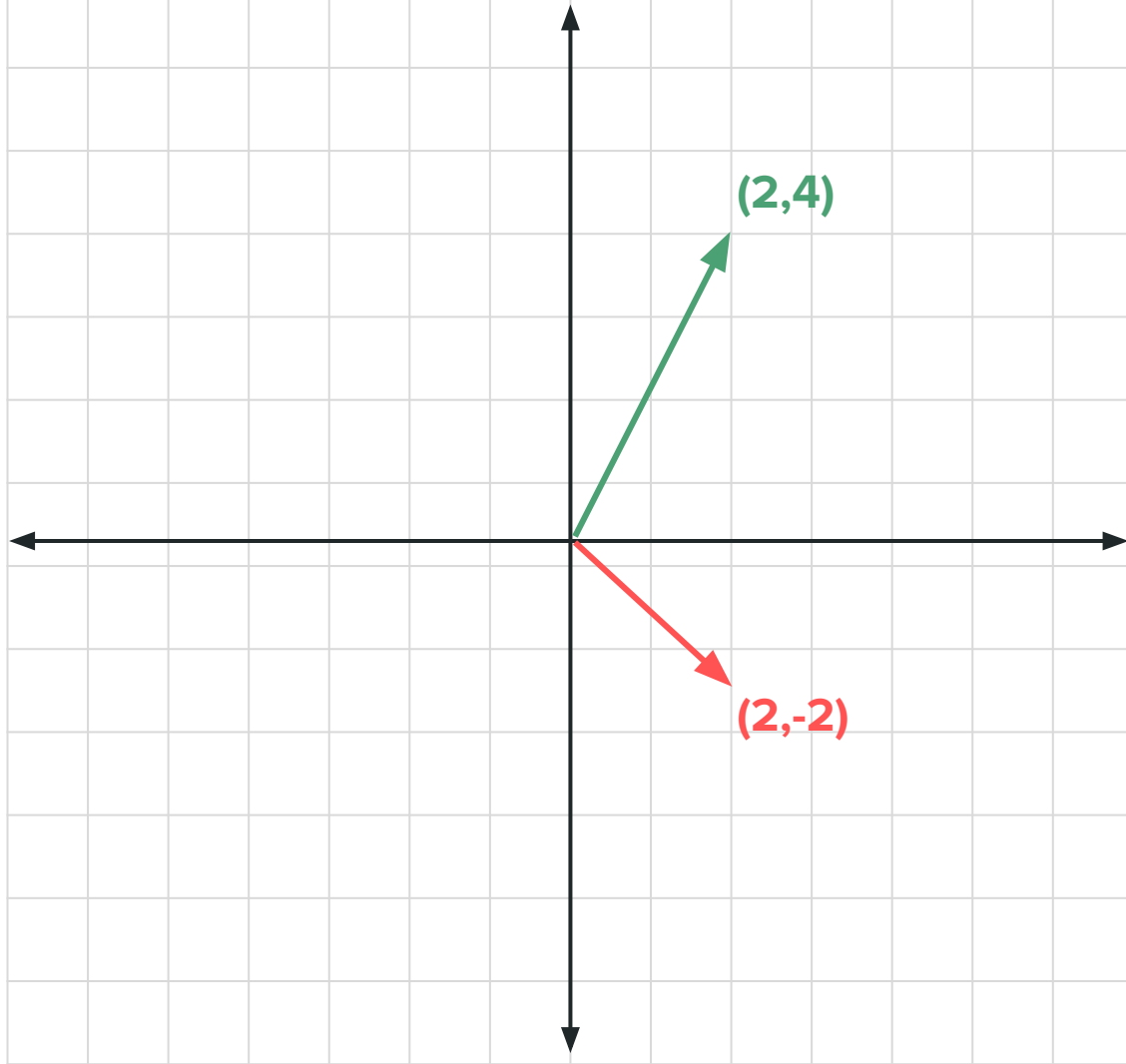
$$c_i = a_i + b_i$$

Elements in a and b with the same index are added together, and their sum stored in c inherits that index

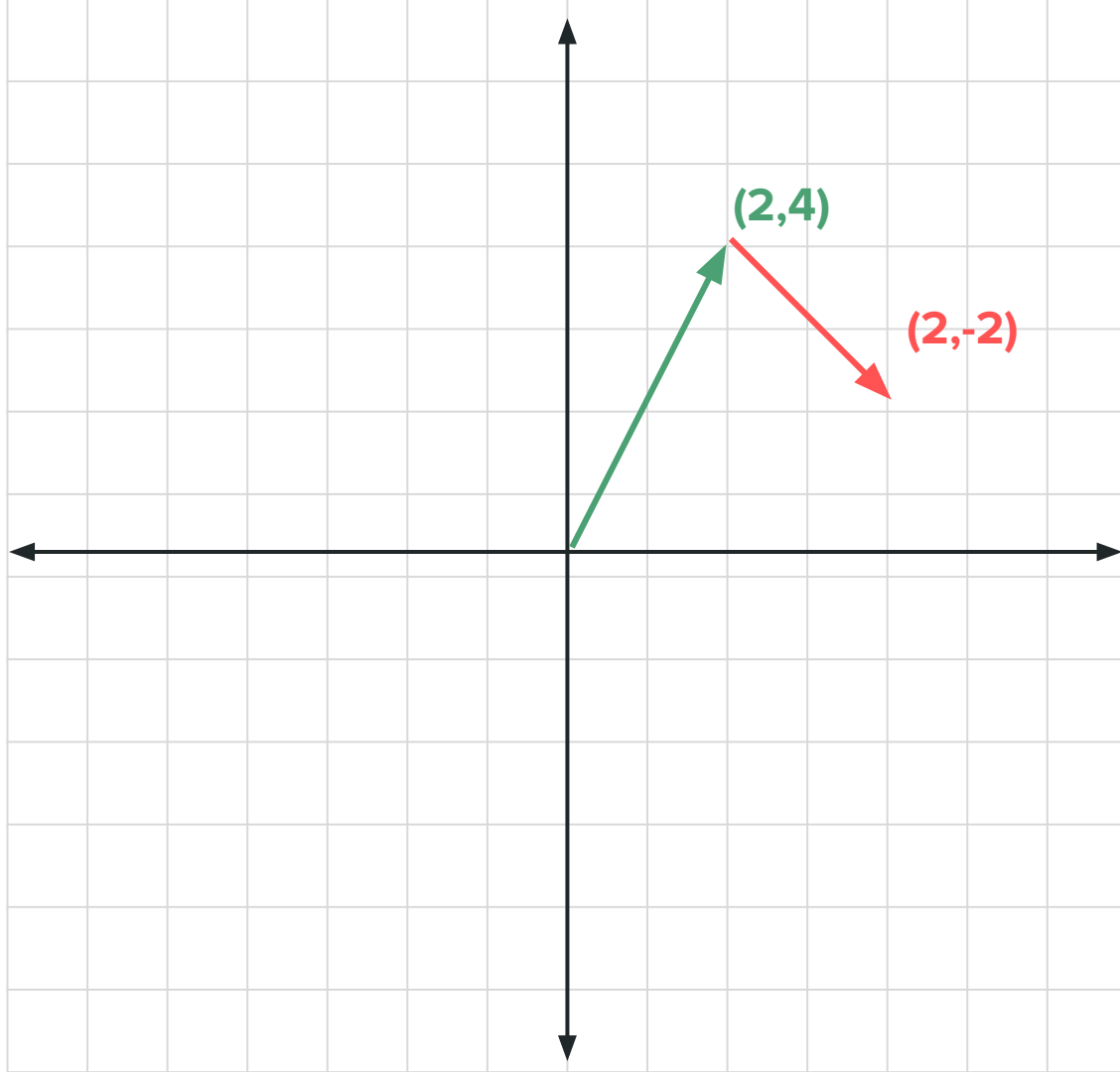
$$[a_1, a_2, a_3] + [b_1, b_2, b_3] = [c_1, c_2, c_3]$$

$$[8, 8, 8] + [-8, -2, -6] = [0, 6, 2]$$

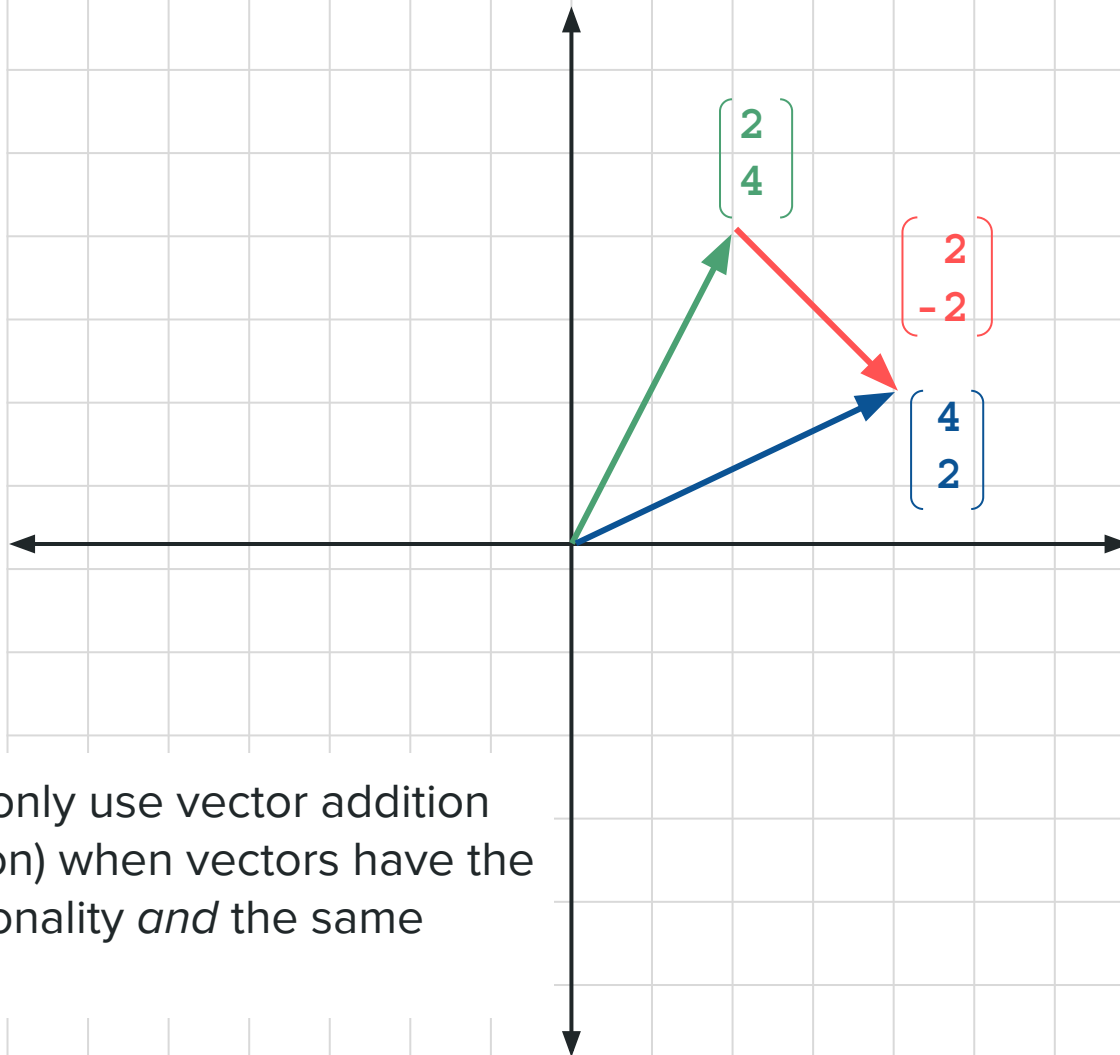
Vector addition



Vector addition



Vector addition



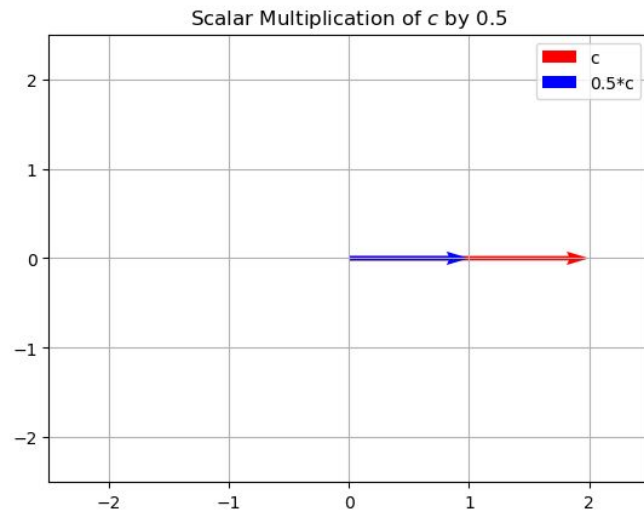
Note: we can only use vector addition (and subtraction) when vectors have the same dimensionality *and* the same orientation.

You can multiply vectors with numbers

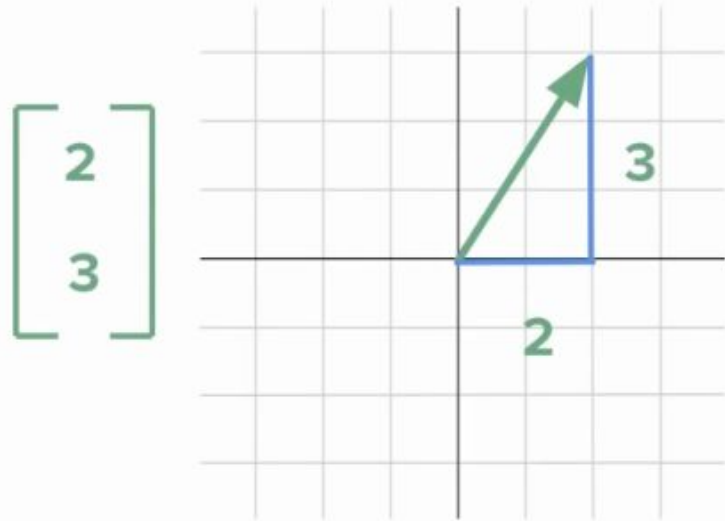
- When we multiply a vector with a number k , we multiply all of the elements in the vector $\mathbf{v}$ by k
- This has the effect of changing the scale (length) of the vector by a factor of k , so we call this number a scalar

$$k * [v_1, v_2] = [kv_1, kv_2]$$

$$0.5 * [2, 0] = [1, 0]$$

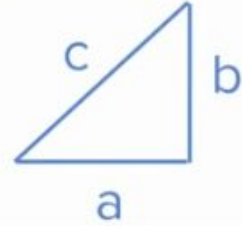


Vector Property: **Length**



- The length of a vector can also be considered the **magnitude** or **norm**.
- The **norm** is a measure of the size or length of a vector or matrix in linear algebra.
- Norm is a generalization of the concept of the magnitude of a vector in Euclidean space.

Pythagorean Theorem



$$c = \sqrt{a^2 + b^2}$$

$$\text{Length of } \mathbf{x} = \sqrt{2^2 + 3^2}$$

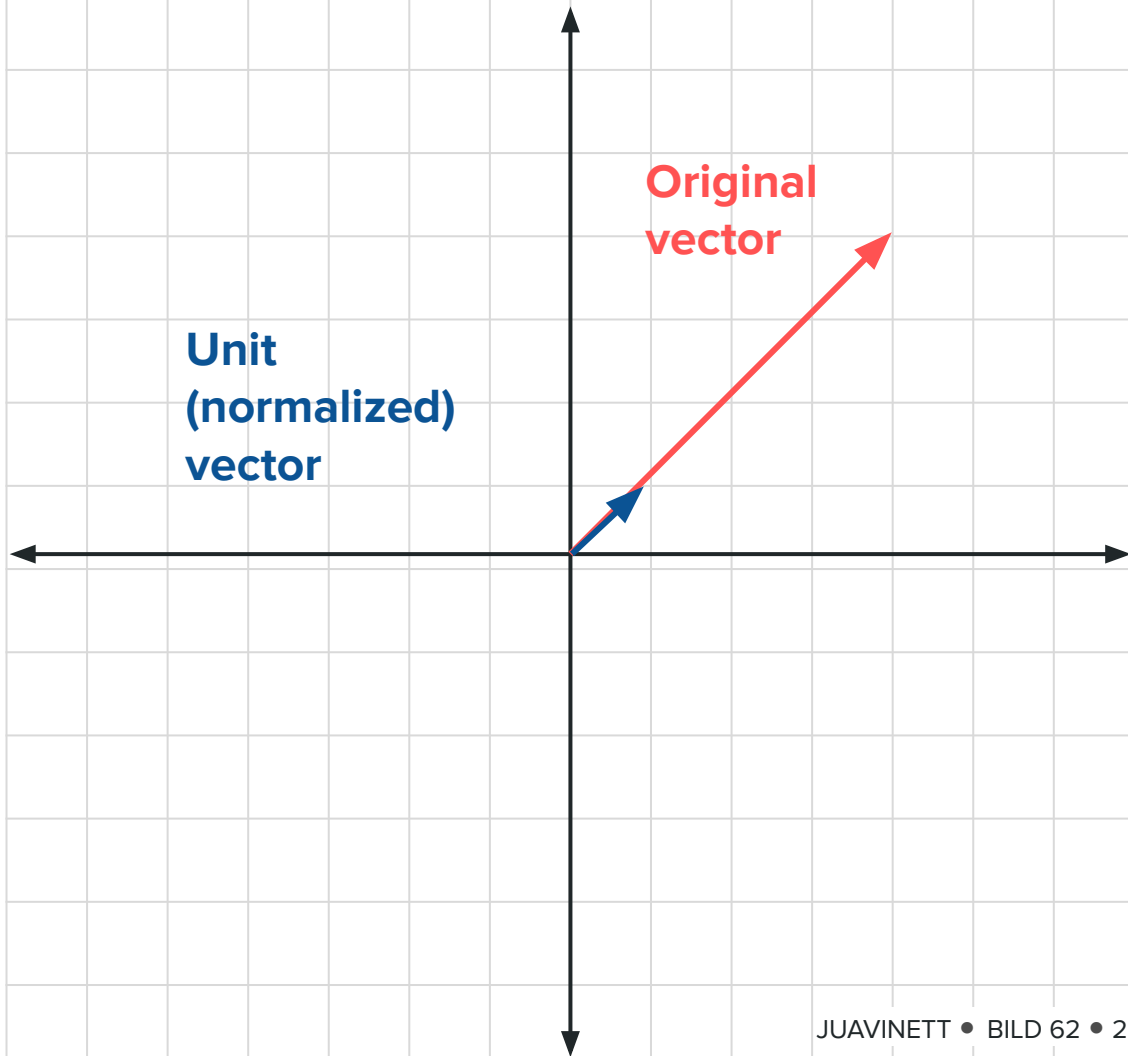
$$\|\mathbf{x}\| = \sqrt{13}$$

//

Vector Normalization

- Dividing a vector by its magnitude to generate a unit vector (with magnitude 1)

$$\tilde{\mathbf{x}} = \frac{\mathbf{x}}{||\mathbf{x}||}$$



Python supports element-wise multiplication

- Not a valid operation in linear algebra, but when you use the `*` operator to multiply two vectors in python code, you perform element-wise multiplication
- **Vectors must have the same dimensionality**

$$[a_1, a_2, a_3] * [b_1, b_2, b_3] = [a_1 b_1, a_2 b_2, a_3 b_3]$$

$$[2, 2, 2] * [1, 2, 3] = [2, 4, 6]$$

- When we perform element-wise multiplication between two vectors, *the resulting vector inherits the shape of the original vectors*

Vector Multiplication Changes the Vector's Shape

- To multiply two vectors you must use one column vector and one row vector
- If both of your vectors have the same orientation, you must *transpose* one of them before you multiply your vectors together
- The order of multiplication matters!

Multiplication Order	Product Dimensions	Product Name	Resulting Dimensions
Row x Column	$(1, N) \times (N, 1)$	Inner Product	$(1,1)$
Column x Row	$(M, 1) \times (1, N)$	Outer Product	(M,N)

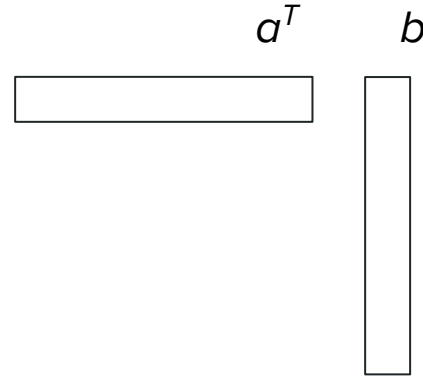
The **dot product**: a single number that provides information about the relationship between two vectors or matrices

Dot operator

You might also see
 $a^T b$ or $\langle a, b \rangle$

transpose

$$a \cdot b = \sum_{i=1}^n a_i b_i$$



a = 1st vector

b = 2nd vector

n = dimension of the vector space

a_i = component of vector a

b_i = component of vector b

In other words, multiply the corresponding elements of two vectors, and then sum over all of the individual products.

In other *other* words, element-wise multiplication & sum.

Example calculations with unit vectors

- A vector with itself:

$$[1, 0]^T \cdot [1, 0] = (1 \times 1) + (0 \times 0) = 1$$

- Perpendicular vectors;

$$[1, 0]^T \cdot [0, 1] = (1 \times 0) + (0 \times 1) = 0$$

- Anti-parallel vectors:

$$[1, 0]^T \cdot [-1, 0] = (-1 \times 1) + (0 \times 0) = -1$$

- For unit vectors, this operation outputs values in the range:

$$-1 \leq \text{value} \leq 1$$

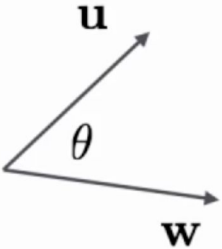
- The value depends on the angle between the vectors
- For arbitrary vectors with lengths p and q , the output will fall in the range:

$$-pq \leq \text{value} \leq pq$$

The **dot product**: a single number that provides information about the relationship between two vectors or matrices

Another way to write this...

Gives intuition for why dot product is 0 when angle is 90! (because $\cos(90) = 0$!)

$$\mathbf{u} \cdot \mathbf{w} = \|\mathbf{u}\| \|\mathbf{w}\| \cos(\theta)$$


The diagram illustrates the geometric interpretation of the dot product. It shows two vectors, $\mathbf{u}$ and $\mathbf{w}$, originating from a common point. Vector $\mathbf{u}$ points upwards and to the right, while vector $\mathbf{w}$ points downwards and to the right. The angle between the two vectors is labeled θ .

The **outer product** results in a matrix

- If we have an m -dimensional vector $\mathbf{u}$ and an n -dimensional vector $\mathbf{v}$:

$$\mathbf{u} = \begin{bmatrix} u_1 \\ u_2 \\ \vdots \\ u_m \end{bmatrix}, \quad \mathbf{v} = \begin{bmatrix} v_1 \\ v_2 \\ \vdots \\ v_n \end{bmatrix}$$

u v^T

- Their outer product is the matrix made by multiplying each element of $\mathbf{u}$ with each element of $\mathbf{v}$:

$$\mathbf{u} \otimes \mathbf{v} = \mathbf{A} = \begin{bmatrix} u_1 v_1 & u_1 v_2 & \dots & u_1 v_n \\ u_2 v_1 & u_2 v_2 & \dots & u_2 v_n \\ \vdots & \vdots & \ddots & \vdots \\ u_m v_1 & u_m v_2 & \dots & u_m v_n \end{bmatrix}$$

Example Outer Product Calculation

$$u = \begin{bmatrix} 0 \\ 1 \\ 2 \end{bmatrix}, v = \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix}$$

$$uv^T = \begin{bmatrix} 0 \\ 1 \\ 2 \end{bmatrix} [1, 2, 3] = A$$

$$A = \begin{bmatrix} 0, 0, 0 \\ 1, 2, 3 \\ 2, 4, 6 \end{bmatrix}$$

Part II: Matrices & Linear Transformations

A brief introduction to matrices

2 x 3 matrix

Matrix A has 2 rows and 3 columns, can be indexed as $A_{i,j}$, where i is the row number and j is the column number.

For example, $A_{1,2} = 4$ and $A_{2,3} = 3$.

$$A = \begin{pmatrix} 2 & 4 & 8 \\ 1 & 7 & 3 \end{pmatrix}$$


Note: Be mindful of indexing differences when translating formulas into Python code!

Special matrices

Square matrices are those with the same number of row and column. $m = n$. For example,

$$A = \begin{pmatrix} 2 & 4 \\ 1 & 5 \end{pmatrix}, b = \begin{pmatrix} 2 & 4 & 8 \\ 1 & 7 & 3 \\ 2 & 5 & 6 \end{pmatrix}$$

Diagonal matrices are square matrices with only the values along the main diagonal are non-zero. For example,

$$C = \begin{pmatrix} 2 & 0 & 0 \\ 0 & 7 & 0 \\ 0 & 0 & 6 \end{pmatrix}$$

Identity matrices are diagonal matrices where all the non-zero values are 1. For example,

$$A = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

Matrix transposition

Transposition flips rows and columns — each row of the original matrix becomes the corresponding column of the new matrix

$$A = \begin{pmatrix} 2 & 4 & 8 \\ 1 & 7 & 3 \end{pmatrix} \longleftrightarrow \begin{matrix} \text{We can move} \\ \text{between these with} \\ \textbf{transposition} \end{matrix} \longleftrightarrow A^T = \begin{pmatrix} 2 & 1 \\ 4 & 7 \\ 8 & 3 \end{pmatrix}$$

Matrix-Vector Multiplication Transforms Vectors

- Matrices and vectors can be multiplied together as long as the inner dimension of the product match:

$$(n \times m) \times (m \times 1) \rightarrow (n \times 1)$$

or

$$(1 \times m) \times (m \times n) \rightarrow (1 \times n)$$

- The result of matrix-vector multiplication is a new **vector** with updated length, direction, and dimensionality

The 2D Rotation Matrix

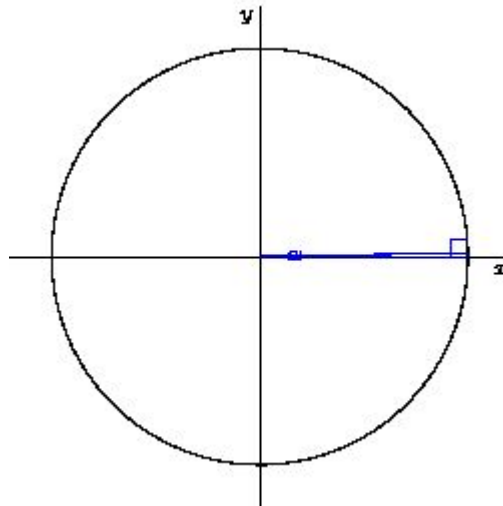
- Special matrix that rotates a vector by an angle θ :

$$R = \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix}$$

For $\theta = 90^\circ$ ($\cos \theta = 0$, $\sin \theta = 1$):

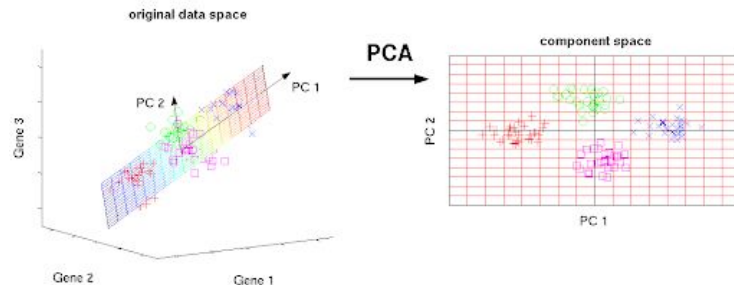
$$v = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$$

$$Rv = \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \begin{bmatrix} (1 * 0) + (0 * -1) \\ (1 * 1) + (0 * 0) \end{bmatrix} = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$$



When is matrix-vector multiplication useful?

- Principal Components Analysis rotates your data to a coordinate system that makes analysis easier
- Linear dynamical models (e.g. population, neural activity) use matrix-vector multiplication to update the state of the system
- Model fitting relies on matrix-vector multiplication to fit parameters



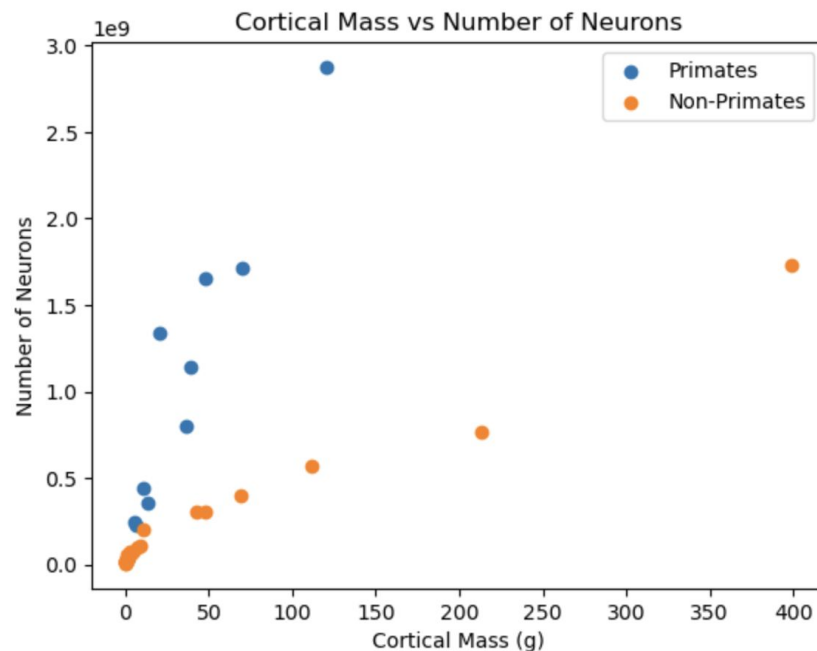
$$\begin{bmatrix} N_1 \\ N_2 \\ N_3 \end{bmatrix}_{t+1} = \begin{bmatrix} F_1 & F_2 & F_3 \\ P_{1 \rightarrow 2} & 0 & 0 \\ 0 & P_{2 \rightarrow 3} & 0 \end{bmatrix} \cdot \begin{bmatrix} N_1 \\ N_2 \\ N_3 \end{bmatrix}_t$$

Leslie Matrix

Part III: Data Analysis Using Linear Algebra

Linear Algebra helps us probe the relationship between variables of interest

- In the Pandas notebook, we plotted *Cortical Mass vs Number of Neurons* variables for our **primate** and **non-primate** datasets
- Looking at the plot, it seems like there is a relationship between these variables!
- We can turn our datasets into vectors and use linear algebra to analyze them



Turing our data into vectors

Dataset 1: Primate

Variable 1: *Cortical Mass*

- 11 Data points → 11-D vector

Variable 2: *Number of Neurons*

- 11 Data points → 11-D vector

Dataset 2: Non-Primate

Variable 1: *Cortical Mass*

- 27 Data points → 27-D vector

Variable 2: *Number of Neurons*

- 27 Data points → 27-D vector

Key Point: Each data point is the measurement of our variable of interest per species in the data set.

Primate contains 11 species and Non-Primate contains 27 species

Pearson's Correlation Coefficient looks for a linear relationship

- We saw that the dot product could be used to measure the alignment between two vectors
- When we compute the normalized dot products between two variables, we can see if they are linearly correlated
 - A correlation coefficient of 1 means the variables are perfectly linearly correlated
 - A correlation coefficient of 0 means there is no linear correlation between variables
 - A correlation coefficient of -1 means that the variables are perfectly anti-correlated linearly
- When we compute the correlation coefficients between *Cortical Mass* and *Number of Neurons* for our **Primate** and **Non-Primate** datasets we get values of 0.95 and 0.99 respectively

What does this tell us about our variables?

Linear Regression fits model parameters to data

- We can try to fit a line to our data to model the linear relationship between our variables *Cortical Mass* and *Number of Neurons*
- We'll assume that *Cortical Mass* is the independent variable in our model and *Number of Neurons* is the dependent variable
- This gives us the linear model:

$$y_{num_neurons} = mx_{cortical_mass} + b$$

Our goal is to fit m and b to our data

Let's solve the regression problem with linear algebra

- We can rewrite our equation as a matrix-vector product:

$$\begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{bmatrix} = \begin{bmatrix} 1 & x_1 \\ 1 & x_2 \\ \vdots & \\ 1 & x_n \end{bmatrix} \begin{bmatrix} b \\ m \end{bmatrix}$$

- Then we just have to solve for the vector $\hat{\beta} = \begin{bmatrix} b \\ m \end{bmatrix}$

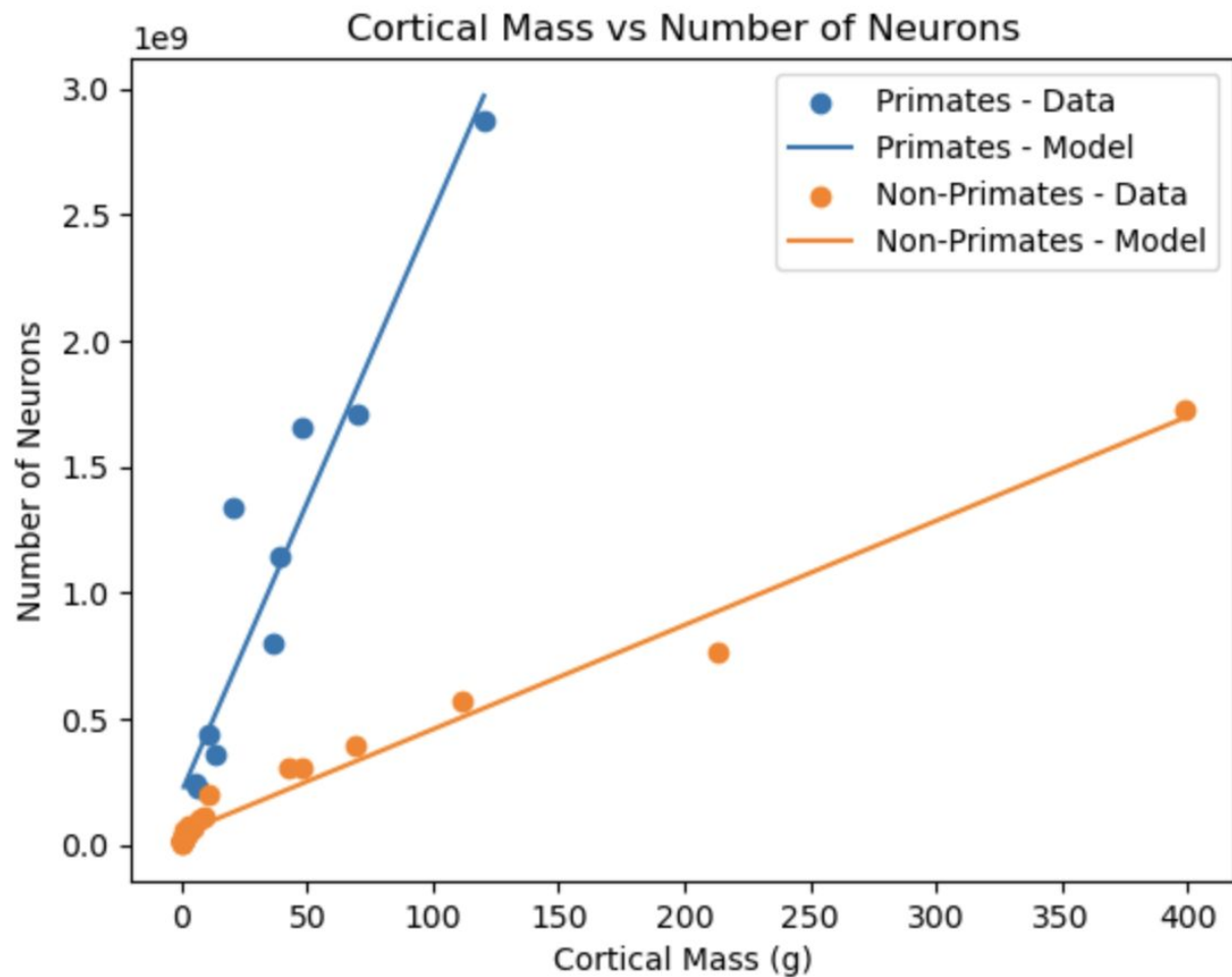
Let's solve the regression problem with linear algebra

- The solution to our regression problem is:

$$\hat{\beta} = (X^T X)^{-1} X^T y$$

- To get the line of best fit for our data, we compute

$$\hat{y} = X\hat{\beta}$$



Additional resources

[Essence of linear algebra - YouTube](#)

[Tutorial 1: Vectors — Neuromatch Academy](#) ← much of this lecture was adopted from these materials!

https://www.youtube.com/watch?v=Ene_TYyTdNM — modeling neural connections as vectors and dot product review

<http://matrixmultiplication.xyz/> — awesome visualization of matrix multiplication!

[Essence of linear algebra - YouTube](#)

Cohen, *Practical Linear Algebra for Data Science*

@ UCSD: MATH 18